

📖 Guia de Consulta Rápida — Análise de Dados em Python

Guia prático e direto ao ponto focado na sintaxe essencial para provas e consultas rápidas. Cobre **Pandas (DataFrames & Series)**, **Tabulação Cruzada (`crosstab`) & `groupby`** Avançado (Aula 11), **Plotagem (Matplotlib & Seaborn)**, **NumPy**, **Estatística Descritiva** e **Distribuições de Probabilidade**.

📖 Sumário

1. Importações Padrão
2. NumPy: Arrays e Vetores
3. Pandas: Series e DataFrames
 - Criando Series e DataFrames
 - Leitura e Inspeção de Arquivos
 - Seleção e Fatiamento (`loc` vs `iloc`)
 - Filtros e Condições Booleanas (MUITO COBRADO)
 - Manipulação de Linhas e Colunas
4. Aula 11: Tabulação Cruzada (`crosstab`) e `groupby` Avançado
 - O que é e Assinatura do `pd.crosstab()`
 - Contagem Simples de Frequência
 - Agregação de Valores (`values` e `aggfunc`)
 - Totais Marginais (`margins=True`)
 - Normalização e Proporções (`normalize`)
 - Visualização com Heatmap (`sns.heatmap`)
 - Gráficos de Barras a partir de Crosstab
 - Groupby Avançado e Múltiplas Agregações (`agg` , `describe` , `reset_index`)
 - Plotando Gráficos Diretamente a partir de Groupby
5. Plotagem e Visualização de Dados (Matplotlib & Seaborn)
 - Configurações Gerais e Estilo
 - Múltiplos Gráficos lado a lado (`subplot`)
 - Gráfico de Pizza (Pie Chart)
 - Gráfico de Barras (Bar Plot)
 - Gráfico de Linhas (Line Plot)
 - Gráfico de Área Empilhada (Stacked Area)
 - Gráfico de Dispersão e Regressão (Scatter / Regplot)
 - Histograma e Distribuição (`displot` / `histplot`)

- Gráfico de Caixa (Boxplot) — Aula 03 & Provas

6. Estatística Descritiva e Probabilidade

- Estatística Descritiva Básica e Quartis (`quantiles` , IQR)
- Distribuição Binomial (`scipy.stats.binom`)
- Distribuição de Poisson (`scipy.stats.poisson`)

7. ⚠ Pegadinhas e Erros Mais Comuns em Provas

1. Importações Padrão

Sempre inicie o código garantindo as seguintes importações:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statistics as st
from scipy.stats import binom, poisson
```

2. NumPy: Arrays e Vetores

Arrays do NumPy são homogêneos (mesmo tipo) e multidimensionais.

```
# Criação

arr = np.array([1, 2, 3, 4, 5])
mat = np.arange(1, 10).reshape(3, 3)      # Matriz 3x3 de 1 a 9
zeros = np.zeros((3, 3))                  # Matriz 3x3 de zeros
ones = np.ones((2, 4))                    # Matriz 2x4 de uns
espacado = np.linspace(0, 100, 5)         # 5 pontos igualmente espaçados de 0 a 100


# Indexação e Fatiamento

arr[0]          # Primeiro elemento
arr[-1]         # Último elemento
arr[1:4]        # Do índice 1 ao 3 (o 4 é exclusivo)
mat[0, :]       # Primeira linha inteira
mat[:, 1]       # Segunda coluna inteira


# Operações Matemáticas

arr.mean()      # Média
arr.std()       # Desvio padrão
arr.sum()       # Soma total
arr.max()       # Valor máximo
arr.min()       # Valor mínimo
```

3. Pandas: Series e DataFrames

Criando Series e DataFrames

```
# Series (1 dimensão rotulada)
dias = ["Seg", "Ter", "Qua", "Qui", "Sex"]
gastos = [40, 55, 30, 70, 65]
s = pd.Series(gastos, index=dias)
s['Ter'] # Retorna 55

# DataFrame (Tabela 2D)
dados = {
    'Nome': ['Ana', 'Bruno', 'Carlos'],
    'Idade': [23, 35, 29],
    'Salario': [4500, 7200, 5100]
}
df = pd.DataFrame(dados)

# A partir de matriz NumPy
df_mat = pd.DataFrame(np.arange(1, 10).reshape(3, 3),
                      columns=['A', 'B', 'C'],
                      index=['L1', 'L2', 'L3'])
```

Leitura e Inspeção de Arquivos

```
# Leitura de CSV (use encoding='latin-1' se houver erro de caracteres)
df = pd.read_csv('caminho/arquivo.csv', encoding='latin-1')

# Inspeção inicial
df.shape          # (n_linhas, n_colunas)
df.head(5)        # 5 primeiras linhas
df.tail(5)         # 5 últimas linhas
df.info()          # Tipos de dados, memória e valores não-nulos
df.describe()      # Resumo estatístico (média, std, min, quartis, max)
df.columns         # Lista dos nomes das colunas
df.dtypes          # Tipo de cada coluna
df['Coluna'].unique() # Valores únicos de uma coluna
df['Coluna'].nunique() # Quantidade de valores únicos
df['Coluna'].value_counts() # Contagem de ocorrências por categoria
df.isnull().sum()  # Quantidade de valores nulos por coluna
```

Seleção e Fatiamento (`loc` vs `iloc`)

Método	Tipo de Busca	Exemplo de Sintaxe
<code>df['Coluna']</code>	Retorna Series da coluna	<code>df['Idade']</code>
<code>df[['Col1', 'Col2']]</code>	Retorna DataFrame com colunas	<code>df[['Nome', 'Salario']]</code>
<code>.loc[linhas, colunas]</code>	Por Rótulo/Nome e condições lógicas	<code>df.loc['Item-1', ['Largura', 'Altura']]</code>
<code>.iloc[linhas, colunas]</code>	Por Índice Inteiro (0, 1, 2...)	<code>df.iloc[0:5, 1:3]</code> (linhas 0 a 4, colunas 1 e 2)

```
# Exemplos com iloc:

df.iloc[0]           # Primeira linha inteira
df.iloc[:, 0]        # Primeira coluna inteira
df.iloc[0:3, 1:4]    # Linhas de 0 a 2 e colunas de 1 a 3
df.iloc[:, :2]       # Todas as linhas, apenas as 2 primeiras colunas
```

Filtros e Condições Booleanas

[!IMPORTANT] Regras de Ouro em Provas:

- Use `&` para `E` (AND) — **NUNCA use a palavra `and`**.
- Use `|` para `OU` (OR) — **NUNCA use a palavra `or`**.
- Use `~` para `NÃO` (NOT / negação).
- Cada condição individual **DEVE** estar entre parênteses: `(df['A'] > 10) & (df['B'] == 'X')`.

```

# Filtro simples
maiores = df[df['Idade'] >= 18]

# Filtro com E (&)
filtro_e = df[(df['Operador'] == 'Op-1') & (df['Comprimento'] > 105)]

# Filtro com OU (|)
filtro_ou = df[(df['Comprimento'] >= 110) | (df['Largura'] >= 53)]

# Filtrar por intervalo
intervalo = df[(df['Comprimento'] > 98) & (df['Comprimento'] < 105)]

# Pertence a uma lista (.isin)
regioes_desejadas = df[df['Regiao'].isin(['Europa', 'Africa'])]

# NÃO pertence a uma lista (~ e .isin)
df_filtrado = df[~df['Regiao'].isin(['America do Norte', 'Asia'])]

# Filtrar e selecionar coluna específica
media_op1 = df[df['Operador'] == 'Op-1']['Comprimento'].mean()

```

Manipulação de Linhas e Colunas

```

# Criar ou alterar coluna calculada
df['Volume'] = df['Comprimento'] * df['Largura'] * df['Altura']

# Remover coluna (axis=1 representa colunas)
df = df.drop('Volume', axis=1)
# ou permanentemente com inplace:
df.drop('Volume', axis=1, inplace=True)

# Remover linhas por índice (axis=0 representa linhas)
df = df.drop(0, axis=0)

# Ordenar DataFrame
df_ordenado = df.sort_values(by='Salario', ascending=False) # Decrescente

```

4. Aula 11: Tabulação Cruzada (**crosstab**) e **groupby**

Avançado

A **Aula 11** foca no cálculo de frequências conjuntas e agregações estatísticas entre variáveis categóricas usando `pd.crosstab()` e `df.groupby()`.

O que é e Assinatura do `pd.crosstab()`

A tabulação cruzada calcula uma tabela de frequência (ou agregação) entre duas ou mais variáveis categóricas.

```
pd.crosstab(  
    index,                # Linhas da tabela (Series ou lista de Series)  
    columns,              # Colunas da tabela (Series ou lista de Series)  
    values=None,          # Coluna numérica para agregar (opcional)  
    aggfunc=None,         # Função de agregação: 'mean', 'sum', 'count', np.me  
n, etc.  
    margins=False,        # Adiciona linha/coluna de total marginal ('All')  
    margins_name='All',   # Nome do total marginal (padrão: 'All')  
    dropna=True,          # Descarta linhas/colunas totalmente NaN  
    normalize=False       # Normalização / porcentagem: False, 'all', 'index',  
    'columns'  
)
```

Parâmetro	Tipo	Descrição
<code>index</code>	Coluna(s)	Variável cujas categorias serão os rótulos das linhas .
<code>columns</code>	Coluna(s)	Variável cujas categorias serão os títulos das colunas .
<code>values</code>	Coluna numérica	Valores sobre os quais a agregação será calculada.
<code>aggfunc</code>	String / Função	Função aplicada a <code>values</code> : <code>'mean'</code> , <code>'sum'</code> , <code>'median'</code> , <code>'std'</code> , <code>'count'</code> , <code>np.mean</code> .
<code>margins</code>	Booleano	<code>True</code> acrescenta a linha e a coluna All com a soma/total.
<code>normalize</code>	<code>False</code> , <code>True</code> , <code>'all'</code> , <code>'index'</code> , <code>'columns'</code>	Divide os valores pelo total geral (<code>'all'</code>), por linha (<code>'index'</code>) ou por coluna (<code>'columns'</code>).

Contagem Simples de Frequência

Conta quantas vezes cada par de categorias aparece junto:

```
# Exemplo da aula: gênero vs nível de escolaridade
ct = pd.crosstab(df['gender'], df['education_level'])
print(ct)

# Saída (contagem direta):
# education_level  college  graduate  high school
# gender
# female           1         3           0
# male             2         0           2
```

Agregação de Valores (`values` e `aggfunc`)

Se quisermos calcular uma métrica (como média da nota, soma de faturamento, total de mortes) em vez de apenas contar:

[!WARNING] Atenção em Prova: Se você passar `values`, é **obrigatório** passar `aggfunc`. Caso contrário, ocorrerá erro `ValueError: values cannot be used without an aggfunc`.

```
# Média da nota (score) por gênero e nível de escolaridade
ct_mean = pd.crosstab(
    index=df['gender'],
    columns=df['education_level'],
    values=df['score'],
    aggfunc='mean' # ou np.mean, 'sum', 'median', 'std', 'max', 'min'
)
print(ct_mean)
```

Totais Marginais (`margins=True`)

Gera automaticamente a linha e a coluna com a soma total de cada categoria:

```
ct_margins = pd.crosstab(df['gender'], df['education_level'], margins=True, margins_name='Total')
print(ct_margins)
```

```
# Saída:
# education_level  college  graduate  high school  Total
# gender
# female           1         3         0         4
# male             2         0         2         4
# Total            3         3         2         8
```

Normalização e Proporções (`normalize`)

Muito cobrado quando a questão pede **"calcule a porcentagem/proporção de..."**:

```
# 1) normalize='all' (ou normalize=True):
# Divide pelo total geral da tabela. A soma de TODAS as células = 1.0 (100%)
ct_norm_total = pd.crosstab(df['gender'], df['education_level'], normalize=True)

# 2) normalize='index':
# Divide pelo total de cada LINHA. Cada linha soma 1.0 (100%)
# Ideal para: "Dentre as mulheres, qual a porcentagem em cada nível?"
ct_norm_linhas = pd.crosstab(df['gender'], df['education_level'], normalize='index')

# 3) normalize='columns':
# Divide pelo total de cada COLUNA. Cada coluna soma 1.0 (100%)
# Ideal para: "Dentre os que fizeram faculdade, qual a porcentagem de cada gênero?"
ct_norm_colunas = pd.crosstab(df['gender'], df['education_level'], normalize='columns')
```

Visualização com Heatmap (`sns.heatmap`)

A melhor forma gráfica de apresentar um `crosstab` é através do mapa de calor do Seaborn:

```
ct = pd.crosstab(df['gender'], df['education_level'], margins=True)

plt.figure(figsize=(8, 6))
sns.heatmap(
    ct,
    annot=True,          # OBRIGATÓRIO: escreve os números dentro de cada célula!
    cmap='coolwarm',     # Paleta de cores ('coolwarm', 'Blues', 'viridis', 'YlGnBu')
    fmt='g'              # Formatação dos números (evita notação científica)
)
plt.title('Mapa de Calor - Educação por Gênero')
plt.show()
```

Gráficos de Barras a partir de Crosstab

Podemos plotar diretamente o DataFrame resultante do `crosstab`:

```
ct = pd.crosstab(df['gender'], df['education_level'])

# Barras agrupadas (lado a lado):
ct.plot(kind='bar', figsize=(8, 5))
plt.title('Distribuição de Nível de Educação por Gênero')
plt.ylabel('Contagem')
plt.show()

# Barras empilhadas (stacked=True):
ct.plot(kind='bar', stacked=True, figsize=(8, 5))
plt.title('Nível de Educação por Gênero (Empilhado)')
plt.ylabel('Total de Alunos')
plt.show()
```

Groupby Avançado e Múltiplas Agregações

O `.groupby()` particiona os dados por chaves e calcula agregações numéricas:

```
# 1) Agrupamento por 1 coluna com média:
df.groupby('Company')[['Sales']].mean()

# 2) Agrupamento por 1 coluna com soma:
df.groupby('Company')[['Sales']].sum()

# 3) Resumo estatístico completo do grupo com .describe():
# Gera colunas: count, mean, std, min, 25%, 50%, 75%, max
df.groupby('Company').describe()

# 4) Múltiplas funções em uma única coluna (.agg):
df.groupby('Company')['Sales'].agg(['mean', 'sum', 'count', 'std', 'max', 'min'])

# 5) Funções diferentes para colunas diferentes (.agg com dicionário):
df.groupby('Company').agg({
    'Sales': ['mean', 'sum'],
    'Score': 'max'
})

# 6) Agrupamento por múltiplas colunas (ex: dia e sexo):
df.groupby(['day', 'sex'])['total_bill'].mean()
```

Plotando Gráficos Diretamente a partir de Groupby

[!IMPORTANT] Dica de Ouro de Prova com Groupby + Seaborn:

O `df.groupby(['day', 'sex'])['total_bill'].mean()` gera uma *Series* com índice multinível. Para passar para o `sns.barplot(data=..., x='day', y='total_bill', hue='sex')`, **sempre use** `.reset_index()` para transformar o resultado de volta em um *DataFrame* padrão com colunas normais!

```
# Exemplo 1 do exercício da Aula 11 (Dataset 'tips'):
# "Obter o valor médio da conta por dia da semana e sexo, e plotar barplot"
df_tips = sns.load_dataset('tips')

df_agrupado = df_tips.groupby(['day', 'sex'])['total_bill'].mean().reset_index()

plt.figure(figsize=(8, 6))
sns.barplot(
    data=df_agrupado,
    x='day',
    y='total_bill',
    hue='sex'
)
plt.title('Valor Médio da Conta por Dia e Sexo do Pagante')
plt.xlabel('Dia da Semana')
plt.ylabel('Valor Médio da Conta ($)')
plt.show()

# Exemplo 2 do exercício da Aula 11 (Dataset 'nobel_prize'):
# "Obter o valor em dinheiro corrigido pago por categoria":
df_nobel.groupby('category')[['prizeAmountAdjusted']].sum()

# "Obter a quantidade de ganhadores do Nobel por continente e plotar barras":
df_continente = df_nobel.groupby('birth_continent')[['category']].count().reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(data=df_continente, x='birth_continent', y='category')
plt.title('Ganhadores de Prêmio Nobel por Continente')
plt.xlabel('Continente de Nascimento')
plt.ylabel('Quantidade de Prêmios')
plt.xticks(rotation=45)
plt.show()
```

5. Plotagem e Visualização de Dados (Matplotlib & Seaborn)

Configurações Gerais e Estilo

```
# Tamanho da figura
plt.figure(figsize=(10, 6))
# ou globalmente para todas:
plt.rcParams['figure.figsize'] = [10, 6]

# Estilos do Seaborn
sns.set_style('darkgrid') # 'whitegrid', 'dark', 'white', 'ticks'
sns.set_context("notebook", font_scale=1.2, rc={"font.size": 12, 'axes.titlesize': 16, 'axes.labelsize': 14})

# Rótulos, títulos e linhas de apoio
plt.title('Título do Gráfico')
plt.xlabel('Eixo X')
plt.ylabel('Eixo Y')
plt.axvline(x=60, color='red', linestyle='--', label='Meta') # Linha vertical
plt.axhline(y=100, color='k', linestyle=':') # Linha horizontal
plt.legend()
plt.tight_layout() # Ajusta margens automaticamente
plt.show() # Exibe o gráfico
```

Múltiplos Gráficos lado a lado (**subplot**)

Sintaxe: `plt.subplot(linhas, colunas, posição)`

```
plt.figure(figsize=(12, 5))

# Primeiro gráfico (esquerda)
plt.subplot(1, 2, 1)
sns.histplot(df_europa['CPI'], bins=15, kde=True, color='blue')
plt.title('Índice de Corrupção - Europa')
plt.xlabel('CPI')

# Segundo gráfico (direita)
plt.subplot(1, 2, 2)
sns.histplot(df_africa['CPI'], bins=15, kde=True, color='orange')
plt.title('Índice de Corrupção - África')
plt.xlabel('CPI')

plt.tight_layout()
plt.show()
```

Gráfico de Pizza (Pie Chart)

Indicado para exibir proporções/porcentagens de partes de um todo.

```
labels = ['Confirmados', 'Prováveis', 'Suspeitos']
sizes = [3351, 453, 200]

plt.figure(figsize=(8, 6))
plt.pie(
    sizes,
    labels=labels,
    autopct='%1.1f%%',      # Formato da porcentagem (ex: 35.4%)
    shadow=True,            # Sombra 3D
    startangle=90           # Ângulo de início
)
plt.axis('equal')         # Garante formato circular perfeito
plt.title('Distribuição dos Casos')
plt.show()
```

Gráfico de Barras (Bar Plot)

Opção A: Direto com Pandas (`df.plot`)

```
# Gráfico simples de barras
df.plot(kind='bar', x='Categoria', y='Valores', color='skyblue')
plt.title('Comparativo de Valores')
plt.show()

# Barras Horizontais
df.plot(kind='barh', x='País', y='PIB')
plt.show()

# Barras agrupadas ou empilhadas a partir de um crosstab
tabela = pd.crosstab(df['Regiao'], df['Ano'], values=df['Total'], aggfunc='sum')

# Agrupadas:
tabela.plot(kind='bar', figsize=(10, 6))

# Empilhadas (Stacked):
tabela.plot(kind='bar', stacked=True, figsize=(10, 6))
plt.title('Evolução por Região (Empilhado)')
plt.ylabel('Total')
plt.show()
```

Opção B: Com Seaborn (`sns.barplot`)

```
# Gráfico com barras divididas por categoria secundária (hue)
plt.figure(figsize=(8, 6))
sns.barplot(
    data=df,
    x='Tipo',
    y='Mortes',
    hue='País',          # Separação por cores
    errorbar=None       # Remove a barra de erro/desvio
)
plt.title('Mortalidade por Tipo e País')
plt.xlabel('Tipo de Caso')
plt.ylabel('Número de Mortes')
plt.show()
```

Gráfico de Linhas (Line Plot)

Ideal para analisar tendências e séries temporais.

```
# Com Matplotlib puro
plt.figure(figsize=(10, 5))
plt.plot(df['Data'], df['Preco_Fechamento'], label='Tesla', color='red', linewidth=2)
plt.plot(df['Data'], df['Media_Movel'], label='Média Móvel', linestyle='--')
plt.title('Evolução do Preço das Ações')
plt.xlabel('Data')
plt.ylabel('Preço (USD)')
plt.legend()
plt.show()

# Direto com Pandas
df.plot(kind='line', x='Ano', y=['PIB_Brasil', 'PIB_Argentina'])
plt.show()

# Com Seaborn
sns.lineplot(data=df, x='Ano', y='Taxa_Crime', hue='Categoria')
plt.show()
```

Gráfico de Área Empilhada (Stacked Area)

Mostra a magnitude cumulativa e a evolução de categorias ao longo do tempo.

```
# Os dados precisam estar tabulados (índice = tempo/eixo X, colunas = categorias)
df_crimes.set_index('Ano')[['Roubo', 'Homicidio', 'Furto']].plot(
    kind='area',
    stacked=True,
    figsize=(10, 6)
)
plt.title('Composição Cumulativa de Crimes ao Longo dos Anos')
plt.xlabel('Ano')
plt.ylabel('Taxa por 100 mil habitantes')
plt.show()
```

Gráfico de Dispersão e Regressão (Scatter / Regplot)

Analisa correlação e relação entre duas variáveis numéricas contínuas.

```
# Seaborn regplot (gráfico de dispersão com reta de regressão linear)
plt.figure(figsize=(8, 6))
sns.regplot(
    data=df,
    x='Idade',
    y='Pressao_Arterial',
    fit_reg=True,      # True adiciona a reta de regressão linear
    color='darkblue',
    scatter_kws={'alpha': 0.6} # Transparência dos pontos
)
plt.title('Pressão Arterial em Função da Idade')
plt.xlabel('Idade (anos)')
plt.ylabel('Pressão Arterial Sistólica (mmHg)')
plt.show()

# Matplotlib puro (apenas dispersão simples)
plt.scatter(df['Idade'], df['Pressao_Arterial'], color='purple')
plt.show()
```

Histograma e Distribuição (`displot` / `histplot`)

Mostra a distribuição de frequências de uma variável contínua.

```
# Histograma com curva de densidade estimada (KDE)
sns.displot(df['CPI_2014'], bins=15, kde=True)
plt.title('Índice de Percepção de Corrupção (2014)')
plt.xlabel('Nível de Corrupção')
plt.ylabel('Contagem')
plt.axvline(60, color='k', linestyle='--', label='Corte 60')
plt.show()

# DICA DE PROVA: Exercício de Histogramas lado a lado filtrados
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Filtros
df_europa = df[df['Regiao'] == 'Europe']
df_africa = df[df['Regiao'] == 'Africa']

# Gráfico 1 (Europa)
sns.histplot(df_europa['CPI_2014'], bins=15, kde=True, ax=axes[0], color='navy')
axes[0].set_title('Índice de Corrupção - Europa')
axes[0].set_xlabel('CPI')

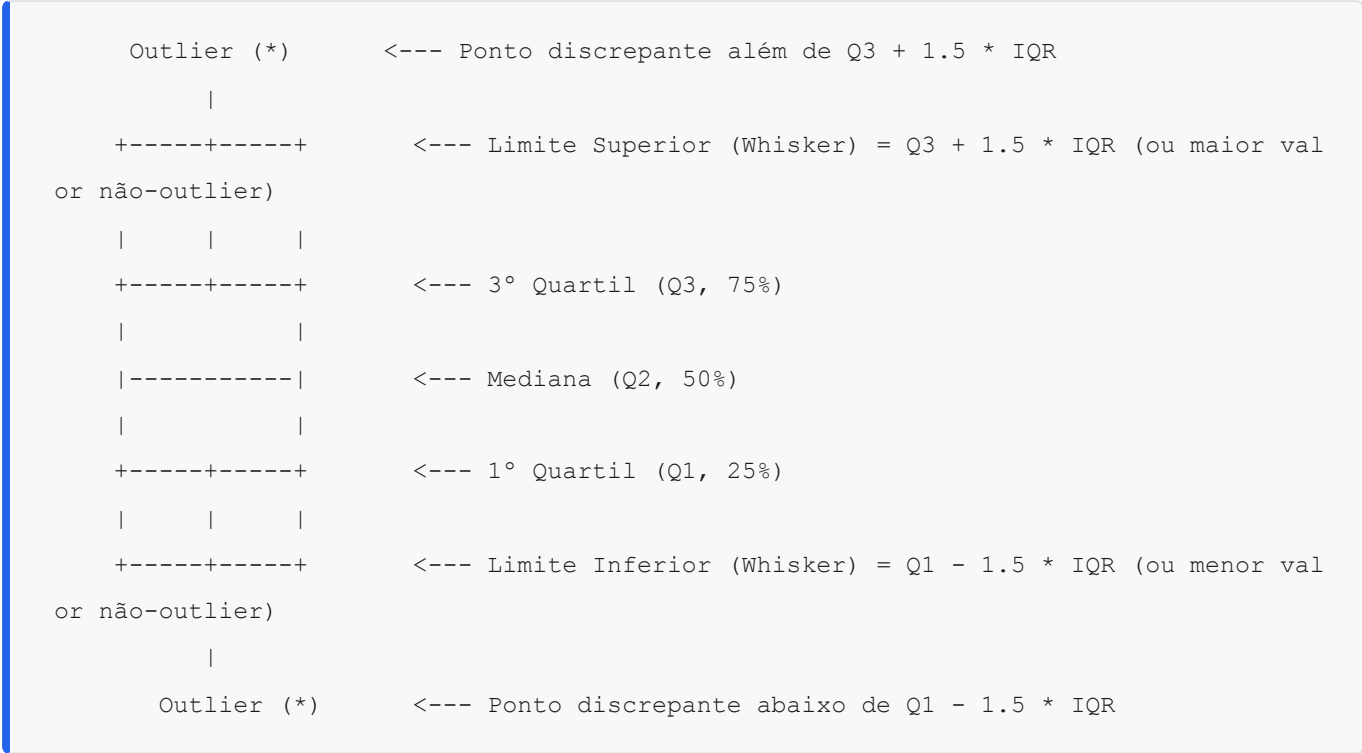
# Gráfico 2 (África)
sns.histplot(df_africa['CPI_2014'], bins=15, kde=True, ax=axes[1], color='crimson')
axes[1].set_title('Índice de Corrupção - África')
axes[1].set_xlabel('CPI')

plt.tight_layout()
plt.show()
```

Gráfico de Caixa (Boxplot) — Aula 03 & Provas

O **Boxplot** (diagrama de caixa) resume a distribuição de uma variável numérica através de cinco medidas estatísticas principais, além de identificar e destacar visualmente os valores discrepantes (*outliers*).

Anatomia Estatística do Boxplot:



- **Q_1 (1º Quartil / 25%)**: 25% dos valores da amostra estão abaixo deste número.
- **Q_2 (Mediana / 50%)**: Valor central da distribuição (50% abaixo, 50% acima).
- **Q_3 (3º Quartil / 75%)**: 75% dos valores da amostra estão abaixo deste número.
- **IQR (Intervalo Interquartil)**: $IQR = Q_3 - Q_1$ (representa a altura da caixa, onde estão os 50% centrais dos dados).
- **Bigodes (Whiskers)**: Limites estabelecidos em $[Q_1 - 1.5 * IQR, Q_3 + 1.5 * IQR]$.
- **Outliers**: Quaisquer valores que fiquem fora dos bigodes (desenhados como círculos ou pontos soltos).

1. Sintaxe no Pandas: `df.boxplot()` (Padrão de Aula e Provas)

```
# Boxplot de uma única coluna numérica
df.boxplot(column='total_bill')
plt.title('Distribuição do Total da Conta')
plt.ylabel('Valor ($)')
plt.show()

# Boxplot agrupado por categoria (column = numérica, by = categórica)
df.boxplot(column='tip', by='day')
plt.title('Gorjeta por Dia da Semana')
plt.suptitle('') # Δ Remove o subtítulo automático feio "Boxplot grouped by day" gerado pelo Pandas!
plt.xlabel('Dia')
plt.ylabel('Gorjeta ($)')
plt.show()

# Outros agrupamentos clássicos de aula:
df.boxplot(column='total_bill', by='sex')
plt.suptitle('')
plt.show()

df.boxplot(column='total_bill', by='size') # por número de pessoas na mesa
plt.suptitle('')
plt.show()
```

2. Padrões Clássicos de Prova (Filtros Prévios + Boxplot)

Nas provas e listas de exercícios (como a ATIV1), o padrão clássico é exigir a **filtragem prévia** com `.i` `sin()` ou operadores lógicos `&` antes de gerar o boxplot:

```
# PADRÃO 1: Filtrar categorias específicas com .isin() antes do boxplot
# Exemplo (ATIV1 Q2): Boxplot do score para gêneros selecionados
generos = ['Animation', 'Comedy', 'Drama', 'Horror', 'Thriller']
df[df['genre'].isin(generos)].boxplot(column='score', by='genre', figsize=(10, 6))
plt.suptitle('')
plt.title('Scores por Gênero Selecionado')
plt.xlabel('Gênero')
plt.ylabel('Score (0 a 10)')
plt.show()

# PADRÃO 2: Filtrar com múltiplas condições numéricas e categóricas
# Exemplo (ATIV1 Q4b): Preço de imóveis por quartos na cidade de Albany (limite até 5
quartos)
df_filtrado = df[(df['city'] == 'Albany') & (df['bed'] <= 5)]
df_filtrado.boxplot(column='price', by='bed', figsize=(9, 5))
plt.suptitle('')
plt.title('Preço de Imóveis por Número de Quartos em Albany (até 5 quartos)')
plt.xlabel('Número de Quartos')
plt.ylabel('Preço (USD)')
plt.show()
```

3. Sintaxe no Seaborn: `sns.boxplot()`

Excelente alternativa para quando o enunciado pedir personalização de paletas, layout horizontal ou subdivisão com `hue`:

```
# Boxplot simples por categoria
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='day', y='total_bill', palette='Set2')
plt.title('Distribuição da Conta por Dia')
plt.show()

# Boxplot com subdivisão por uma terceira variável categórica (hue)
plt.figure(figsize=(10, 6))
sns.boxplot(
    data=df,
    x='day',
    y='total_bill',
    hue='sex',          # Separa homens e mulheres lado a lado em cada dia
    palette='coolwarm'
)
plt.title('Valor da Conta por Dia e Sexo')
plt.show()

# Boxplot Horizontal (basta passar a coluna numérica em x)
sns.boxplot(data=df, x='total_bill', color='lightblue')
plt.title('Boxplot Horizontal de Total da Conta')
plt.show()
```

6. Estatística Descritiva e Probabilidade

Estatística Descritiva Básica e Quartis

```
import statistics as st

# Dados brutos
dados = [10, 15, 12, 14, 15, 18, 20]

media = st.mean(dados)          # Média aritmética
mediana = st.median(dados)      # Mediana (valor central / Q2)
moda = st.mode(dados)          # Moda (valor mais frequente)
variancia = st.variance(dados) # Variância amostral
desvio_pad = st.stdev(dados)   # Desvio padrão amostral

# Quartis com a biblioteca 'statistics' (MUITO COBRADO NA AULA 03):
# Retorna uma lista de 3 valores: [Q1, Q2 (mediana), Q3]
quartis = st.quantiles(df['total_bill']) # Exemplo: [13.30, 17.79, 24.22]
q1 = quartis[0] # 25% dos dados
q2 = quartis[1] # 50% dos dados (mediana)
q3 = quartis[2] # 75% dos dados
iqr = q3 - q1    # Intervalo Interquartil (IQR)

# No Pandas:
df['Coluna'].mean()
df['Coluna'].median()
df['Coluna'].std()
df['Coluna'].quantile(0.25) # 1º quartil (25%)
df['Coluna'].quantile(0.75) # 3º quartil (75%)
df['Coluna'].describe()     # Resumo com count, mean, std, min, 25%, 50%, 75%, max
```

Distribuição Binomial (`scipy.stats.binom`)

Variável discreta: n tentativas independentes, probabilidade de sucesso p .

- `pmf(k, n, p)`: Probabilidade pontual de **exatamente** k sucessos.
- `cdf(k, n, p)`: Probabilidade acumulada de **até** k sucessos ($P(X \leq k)$).

```

from scipy.stats import binom

n = 10 # Tentativas
p = 0.5 # Probabilidade de sucesso

# Probabilidade de exatamente 3 sucessos:  $P(X = 3)$ 
p_3 = binom.pmf(3, n, p)

# Probabilidade de no máximo 3 sucessos:  $P(X \leq 3)$ 
p_ate_3 = binom.cdf(3, n, p)

# Probabilidade de mais de 3 sucessos:  $P(X > 3) = 1 - P(X \leq 3)$ 
p_mais_3 = 1 - binom.cdf(3, n, p)

```

Distribuição de Poisson (`scipy.stats.poisson`)

Ocorrência de eventos em um intervalo fixo com taxa média μ (ou λ).

- `pmf(k, mu)` : Probabilidade de ocorrerem **exatamente k** eventos.
- `cdf(k, mu)` : Probabilidade de ocorrerem **até k** eventos ($P(X \leq k)$).

```

from scipy.stats import poisson

mu = 4 # Média de ocorrências no intervalo

# Probabilidade de ocorrerem exatamente 2 eventos:
p_2 = poisson.pmf(2, mu)

# Probabilidade de ocorrerem até 2 eventos:
p_ate_2 = poisson.cdf(2, mu)

# Probabilidade de ocorrerem mais de 2 eventos:
p_mais_2 = 1 - poisson.cdf(2, mu)

```

7. ⚠ Pegadinhas e Erros Mais Comuns em Provas

Erro Comum	Forma Errada ❌	Forma Correta ✅	Motivo / Explicação
<code>and</code> / <code>or</code> em DataFrames	<code>df[df['A']>5 and df['B']<2]</code>	<code>df[(df['A']>5) & (df['B']<2)]</code>	Em Pandas use operadores bitwise <code>&</code> e <code> </code> .
Esquecer parênteses nos filtros	<code>df[df['A']>5 & df['B']<2]</code>	<code>df[(df['A']>5) & (df['B']<2)]</code>	Em Python o <code>&</code> tem precedência maior que <code>></code> e <code><</code> .
Eixo incorreto no <code>.drop()</code>	<code>df.drop('Coluna')</code>	<code>df.drop('Coluna', axis=1)</code>	O padrão é <code>axis=0</code> (linhas). Colunas exigem <code>axis=1</code> .
<code>values</code> sem <code>aggfunc</code> no <code>crosstab</code>	<code>pd.crosstab(x, y, values=z)</code>	<code>pd.crosstab(x, y, values=z, aggfunc='mean')</code>	Passar <code>values</code> sem <code>aggfunc</code> lança erro no Pandas.
Esquecer <code>.reset_index()</code> no <code>groupby</code>	<code>sns.barplot(df.groupby('A')['B'].mean())</code>	<code>sns.barplot(data=df.groupby('A')['B'].mean().reset_index(), x='A', y='B')</code>	Seaborn precisa acessar as colunas pelo nome.
<code>annot=True</code> no <code>sns.heatmap</code>	<code>sns.heatmap(ct)</code> (fica sem números)	<code>sns.heatmap(ct, annot=True, fmt='g')</code>	<code>annot=True</code> exibe os valores numéricos em cada célula.
Normalização errada no <code>crosstab</code>	Usar <code>normalize=True</code> achando que normaliza por linha	<code>normalize='index'</code> (por linha) ou <code>'columns'</code> (por coluna)	<code>True</code> (ou <code>'all'</code>) divide pelo total geral da tabela inteira.
<code>loc</code> vs <code>iloc</code>	<code>df.loc[0:2]</code> (procurando rótulos)	<code>df.iloc[0:2]</code> (procurando posições)	<code>loc</code> usa rótulos (nomes), <code>iloc</code> usa números inteiros.
Erro de acentuação no CSV	<code>pd.read_csv('dados.csv')</code>	<code>pd.read_csv('dados.csv', encoding='latin-1')</code>	Arquivos em português costumam exigir <code>latin-1</code> ou <code>iso-8859-1</code> .
Alterar DataFrame sem salvar	<code>df.drop('Col', axis=1)</code>	<code>df = df.drop('Col', axis=1)</code> ou <code>inplace=True</code>	Métodos do Pandas retornam uma cópia, a menos que reatribuídos.
Gráficos sobrepostos	Executar <code>plt.plot()</code> em células diferentes	Usar <code>plt.show()</code> ou <code>plt.subplot()</code>	Sem <code>plt.show()</code> , o Matplotlib pode acumular curvas na mesma figura.
Subplot fora de ordem	<code>plt.subplot(1, 2, 0)</code>	<code>plt.subplot(1, 2, 1)</code>	O índice do <code>subplot</code> começa em 1 (1-indexado).
Inverter <code>column</code> e <code>by</code> no <code>df.boxplot</code>	<code>df.boxplot(column='tip', by='day')</code>	<code>df.boxplot(column='tip', by='day')</code>	<code>column</code> é o valor numérico (eixo Y) e <code>by</code> é a categoria (eixo X).
Subtítulo duplicado/colidindo no Boxplot	Não tratar o título automático	Adicionar <code>plt.suptitle('')</code>	Pandas adiciona automaticamente um título <code>Boxplot</code> .

Erro Comum	Forma Errada ❏	Forma Correta ❏	Motivo / Explicação
			<code>xplot grouped by...</code> que sobrepõe <code>plt.title()</code> .
Esquecer filtro <code>.isin()</code> antes do <code>boxplot</code>	Tentar filtrar dentro do método <code>.boxplot()</code>	<code>df[df['col'].isin([...])].boxplot(...)</code>	O método <code>.boxplot()</code> do Pandas não possui parâmetro de filtro; filtre o DataFrame antes.

❏ **Dicas Práticas Rápidas de Prova:**

- **Filtro condicional:** `df[(df['Operador'] == 'Op-1') & (df['Comprimento'] > 105)]`
- **Boxplot com agrupamento e filtro:** `df[df['genre'].isin(['Drama', 'Action'])].boxplot(column='score', by='genre'); plt.suptitle('')`
- **Quartis com statistics (Aula 03):** `st.quantiles(df['coluna'])` retorna `[Q1, Q2, Q3]`
- **Crosstab com agregação e margens:** `pd.crosstab(df['gender'], df['education_level'], values=df['score'], aggfunc='mean', margins=True)`
- **Groupby com plot no Seaborn:** `sns.barplot(data=df.groupby(['dia', 'sexo'])['total'].mean().reset_index(), x='dia', y='total', hue='sexo')`
- **Heatmap formatado:** `sns.heatmap(tabela_crosstab, annot=True, cmap='coolwarm')`